

Neural Networks & Backpropagation

CS224N Week 2: Gradient-Based Learning

Stanford CS224N

Key Concept

The Big Picture: Neural networks learn by composing simple operations into complex functions. Backpropagation is just the chain rule applied efficiently to compute gradients for learning.

Contents

1	Neural Network Fundamentals	3
1.1	The Revolutionary Idea	3
1.2	Architecture Overview	3
1.3	Layer Computation	3
2	Activation Functions	4
2.1	Visual Comparison	4
2.2	Quick Reference	4
3	Matrix Calculus Essentials	5
3.1	From Scalars to Matrices	5
3.2	Chain Rule: The Heart of Backprop	5
3.3	Essential Jacobians (Memorize These!)	5
3.4	The Shape Convention	5
4	Backpropagation: Efficient Gradient Computation	6
4.1	The Two Core Ideas	6
4.2	Computational Graph	6
4.3	Forward & Backward Pass	6
4.4	Node-Level Chain Rule	6
4.5	Gradient Flow Patterns	7
4.6	Worked Example: $f(x, y, z) = (x + y) \cdot \max(y, z)$	7
5	Efficiency & Practical Considerations	8
5.1	Computational Complexity	8
5.2	The Upstream Gradient Trick	8
5.3	Modern Framework Architecture	8
5.4	Implementation Pattern	8
6	Gradient Checking	9
6.1	Numerical Approximation	9
6.2	When to Use	9

7	Summary & Quick Reference	10
7.1	Cheat Sheet	10
7.2	What You Should Be Able to Do	10
7.3	Looking Ahead	10

1 Neural Network Fundamentals

1.1 The Revolutionary Idea

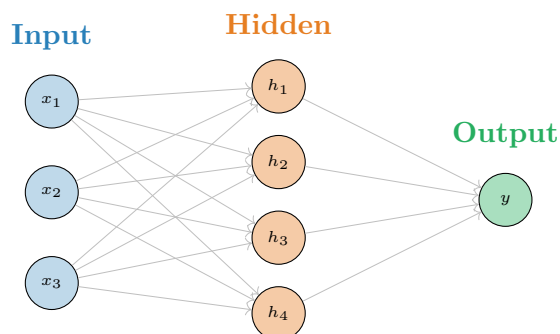
Why Neural Networks Are Powerful

Traditional ML: You design features $\rightarrow$ One classifier $\rightarrow$ Output

Neural Networks: Raw input $\rightarrow$ **Self-organizing layers** $\rightarrow$ Output

The network **automatically learns** useful intermediate representations!

1.2 Architecture Overview



1.3 Layer Computation

Every layer does two things:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b} \quad (\text{Linear: weighted sum}) \quad (1)$$

$$\mathbf{h} = f(\mathbf{z}) \quad (\text{Nonlinear: activation}) \quad (2)$$

where $W \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$, f = activation function

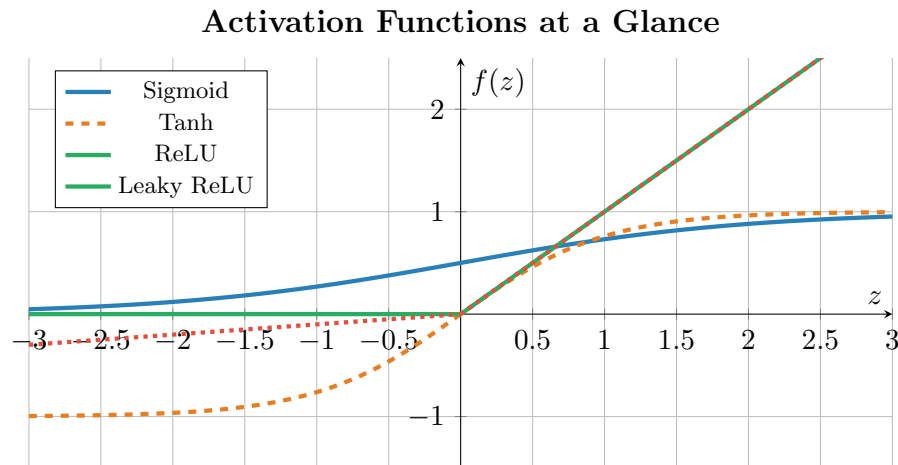
Why Nonlinearity Matters

Without $f(\cdot)$: $W^{(2)}(W^{(1)}\mathbf{x}) = \underbrace{(W^{(2)}W^{(1)})}_{\text{single matrix}}\mathbf{x}$

Multiple layers collapse into one! **No representational power gained.**

2 Activation Functions

2.1 Visual Comparison



2.2 Quick Reference

Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$

- Range: $(0, 1)$
- Use: Output probabilities
- Issue: Always positive

Tanh $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

- Range: $(-1, 1)$
- Use: Hidden layers (Assignment 3!)
- Better: Zero-centered

ReLU $\text{ReLU}(z) = \max(0, z)$

- Fast: No exponentials
- Simple gradient: 0 or 1
- Issue: Dead for $z < 0$

Leaky ReLU $f(z) = \max(\alpha z, z)$

- $\alpha \approx 0.01$
- Never fully dead
- Modern: Swish, GELU

Practical Insight

Default choice: ReLU for hidden layers (fast, simple gradients)

Modern Transformers: GELU or Swish (smooth, never fully dead)

Output layer: Sigmoid for binary, Softmax for multi-class

3 Matrix Calculus Essentials

Key Concept

Mantra: Matrix calculus = Single-variable calculus, but with matrices

3.1 From Scalars to Matrices

Type	Math	Gradient Shape
Scalar $\rightarrow$ Scalar	$f : \mathbb{R} \rightarrow \mathbb{R}$	$\frac{df}{dx} \in \mathbb{R}$
Vector $\rightarrow$ Scalar	$f : \mathbb{R}^n \rightarrow \mathbb{R}$	$\nabla f \in \mathbb{R}^n$ (gradient)
Vector $\rightarrow$ Vector	$\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$	$J \in \mathbb{R}^{m \times n}$ (Jacobian)

3.2 Chain Rule: The Heart of Backprop

Single Variable: $\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$
Multivariable: $\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ (multiply Jacobians)

3.3 Essential Jacobians (Memorize These!)

1. **Element-wise activation:** $\mathbf{h} = f(\mathbf{z})$ where $h_i = f(z_i)$

$$\frac{\partial \mathbf{h}}{\partial \mathbf{z}} = \text{diag}(f'(z_1), f'(z_2), \dots, f'(z_n))$$

2. **Linear transform:** $\mathbf{z} = W\mathbf{x} + \mathbf{b}$

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = W, \quad \frac{\partial \mathbf{z}}{\partial \mathbf{b}} = I$$

3. **Dot product:** $s = \mathbf{u}^T \mathbf{h}$

$$\frac{\partial s}{\partial \mathbf{h}} = \mathbf{u}^T \quad \text{or} \quad \mathbf{u} \text{ (shape convention)}$$

3.4 The Shape Convention

Important for Implementation

Math: Gradients are Jacobians (can be huge matrices)

Code: Reshape gradients to match parameter shapes

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla_{\theta} \mathcal{L}$$

If θ is $n \times m$ matrix $\rightarrow$ gradient should also be $n \times m$

4 Backpropagation: Efficient Gradient Computation

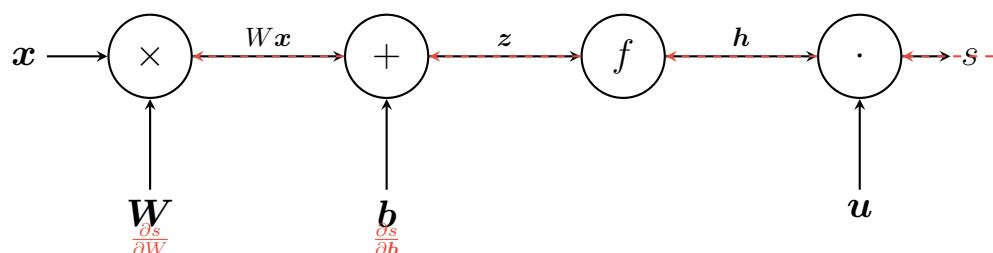
4.1 The Two Core Ideas

Key Concept

Backpropagation = **Chain Rule** + **Memoization**

1. Use chain rule to decompose gradients
2. Store intermediate results (never recompute!)

4.2 Computational Graph



4.3 Forward & Backward Pass

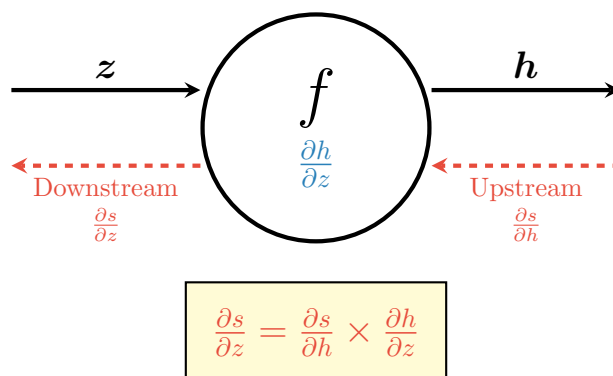
Forward Pass

1. Topological sort
2. Compute outputs
3. **Store values**

Backward Pass

1. Start: $\frac{\partial s}{\partial s} = 1$
2. Reverse order
3. **Reuse stored values**

4.4 Node-Level Chain Rule



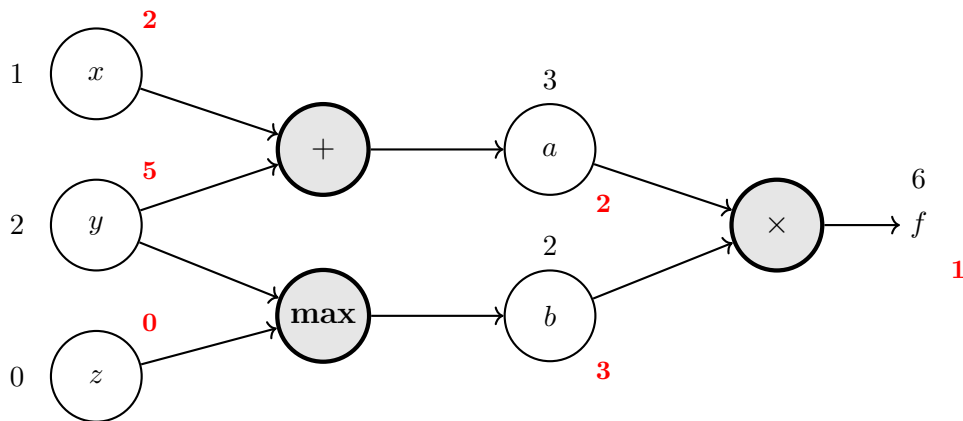
4.5 Gradient Flow Patterns

How different operations route gradients:

Operation	Behavior	Gradient Rule
$c = a + b$	Distributor	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c}, \frac{\partial L}{\partial b} = \frac{\partial L}{\partial c}$
$c = a \times b$	Switcher	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c} \cdot b, \frac{\partial L}{\partial b} = \frac{\partial L}{\partial c} \cdot a$
$c = \max(a, b)$	Router	Gradient $\rightarrow$ winner, 0 $\rightarrow$ loser

4.6 Worked Example: $f(x, y, z) = (x + y) \cdot \max(y, z)$

Given: $x = 1, y = 2, z = 0$



Gradient Computation

1. Start: $\frac{\partial f}{\partial f} = 1$
2. **Multiply ($\times$):** Swap values $\rightarrow \frac{\partial f}{\partial a} = b = 2, \frac{\partial f}{\partial b} = a = 3$
3. **Max:** Route to winner $\rightarrow \frac{\partial f}{\partial y} = 3$ (via max), $\frac{\partial f}{\partial z} = 0$
4. **Add (+):** Distribute $\rightarrow \frac{\partial f}{\partial x} = 2, \frac{\partial f}{\partial y} = 2$ (via add)
5. **Branch at y :** Sum paths $\rightarrow \frac{\partial f}{\partial y} = 3 + 2 = 5$ ✓

5 Efficiency & Practical Considerations

5.1 Computational Complexity

Key Theorem

If forward pass is $O(n)$, backward pass is also $O(n)$.
Each operation's local gradient computed once and reused via chain rule.

5.2 The Upstream Gradient Trick

✗ **Inefficient:** Compute each gradient from scratch

$$\frac{\partial s}{\partial \mathbf{b}}, \quad \frac{\partial s}{\partial W}, \quad \frac{\partial s}{\partial \mathbf{x}} \quad (\text{redundant work!})$$

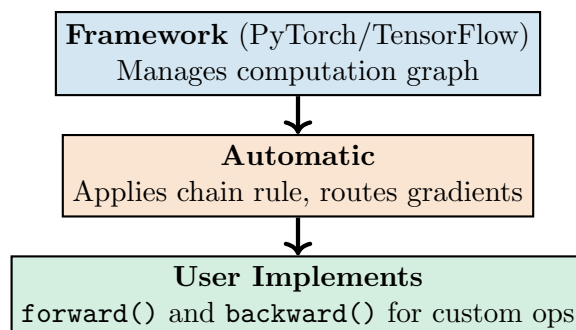
✓ **Efficient:** Compute shared part once

$$\delta = \frac{\partial s}{\partial \mathbf{h}} \cdot \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \quad (\text{upstream gradient})$$

Then reuse:

$$\begin{aligned} \frac{\partial s}{\partial \mathbf{b}} &= \delta \\ \frac{\partial s}{\partial W} &= \delta^T \mathbf{x}^T \\ \frac{\partial s}{\partial \mathbf{x}} &= W^T \delta^T \end{aligned}$$

5.3 Modern Framework Architecture



5.4 Implementation Pattern

```
class MultiplyGate:
    def forward(self, x, y):
        self.x = x # Cache for backward!
        self.y = y
        return x * y

    def backward(self, grad_output):
        # Local gradient computation
        grad_x = grad_output * self.y
        grad_y = grad_output * self.x
        return grad_x, grad_y
```


6 Gradient Checking

6.1 Numerical Approximation

Key Concept

Verify your gradients numerically:

$$\frac{\partial f}{\partial x} \approx \frac{f(x+h) - f(x-h)}{2h}$$

where $h \approx 10^{-4}$ for neural networks

Two-Sided is Better!

Two-sided: Error $\sim O(h^2)$ ✓ Use this!

One-sided: Error $\sim O(h)$ ✗ Less accurate

6.2 When to Use

- ✓ When implementing custom layers
- ✓ Debugging gradient bugs
- ✗ During regular training (too slow!)

Procedure:

1. Compute g_{analytic} using backprop
2. Compute g_{numeric} using finite differences
3. Check: $|g_{\text{analytic}} - g_{\text{numeric}}| < 10^{-2}$

7 Summary & Quick Reference

Core Takeaways

1. **Neural networks** = Cascades of $\mathbf{z} = W\mathbf{x} + \mathbf{b}$, $\mathbf{h} = f(\mathbf{z})$
2. **Nonlinearity** is essential (otherwise layers collapse)
3. **Backpropagation** = Chain rule + don't recompute
4. **Matrix calculus** = Single-variable calculus with matrices
5. **Computational graphs** enable automatic differentiation

7.1 Cheat Sheet

Activation Gradients

- ReLU: $f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$
- Sigmoid: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- Tanh: $\tanh'(z) = 1 - \tanh^2(z)$

Common Jacobians

- $\mathbf{z} = W\mathbf{x} + \mathbf{b}$: $\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = W$
- $s = \mathbf{u}^T \mathbf{h}$: $\frac{\partial s}{\partial \mathbf{h}} = \mathbf{u}^T$
- Element-wise: Diagonal matrix

Gradient Flow

- Addition: Distributes
- Multiplication: Swaps
- Max: Routes to winner
- Branching: Sum gradients

Implementation

- Store values in forward
- Use stored values in backward
- Match gradient shape to parameters
- Check with numerical gradients

7.2 What You Should Be Able to Do

✓ Draw computation graphs

✓ Compute forward pass

✓ Derive gradients by hand

✓ Implement forward/backward

✓ Check gradients numerically

7.3 Looking Ahead

Future Topics:

- Vanishing/exploding gradients (why deep learning was hard)
- Optimization algorithms (Adam, RMSprop)

- Regularization (dropout, batch norm)
- Advanced architectures (RNNs, Transformers)

Final Thought

High school students can build models without knowing this math (which is great!).

But understanding the foundations makes you **debug faster, innovate better, and understand** when things go wrong.